

CHAPTER 1: INTRODUCTION

Multipliers, as fundamental components of these systems, significantly impact power consumption and computational speed. However, traditional multipliers demand substantial power and area, leading to challenges in designing energy efficient circuits. Approximate computing has emerged as a promising paradigm to address these challenges by leveraging the inherent error tolerance in many applications. By trading off a small amount of accuracy, approximate multipliers can achieve significant reductions in power consumption, delay, and silicon area. Among the various techniques, truncation is a widely used approximation method that discards less significant bits to minimize power and complexity. However, fixed truncation schemes lack flexibility, leading to suboptimal accuracy for varying application requirements. This project explores the design and implementation of a low-power and highspeed approximate multiplier using reconfigurable truncation. The proposed design leverages transmission gate logic, known for its low-power characteristics and high-speed operation, to achieve efficient multiplication. The reconfigurable truncation technique enables dynamic adjustment of speed and power consumption, making the multiplier adaptable to different application scenarios. By employing transmission gate logic, the multiplier benefits from reduced power dissipation, lower leakage currents, and enhanced speed compared to conventional CMOS logic designs. The reconfigurable truncation mechanism further enhances the design's versatility, allowing a balanced trade-off between accuracy and energy efficiency.

This project aims to provide an innovative solution to meet the growing demand for power-efficient arithmetic units while maintaining high computational accuracy. The proposed approximate multiplier is expected to contribute to the results are not necessary. Multipliers are among the most critical arithmetic functional units in many applications, such as digital signal processing (DSP), computer vision, multimedia processing, image recognition,

and artificial intelligence. Those applications commonly need numerous multiplications that result in huge power consumption. The high-power consumption is a challenge for implementing those applications, especially on mobile devices. Therefore, many studies have proposed techniques to reduce the power consumption of multiplier circuits. One solution to reduce the power consumption of a multiplier is to approximate multiplication if the targeted applications allow error tolerance, or in other words, if they are related to human senses. Due to the human sensory limitations, such as limited viewing spectrum and hearing range, the accurate computing editor coordinating the review of this manuscript and approving it for publication was Mario Donato Marino . The approximate multipliers sacrifice accuracy in exchange for the reduction of cell area, timing delay, and power consumption.

Approximate multipliers can be categorized into two types. The first type is to control the timing path of the multiplier, which can be achieved by using the dynamic voltage scaling. If a lower voltage is applied to a multiplier, the delay of the critical path will increase. Therefore, when the violation of the timing path happens, the errors occur, generating approximate results. The second type is to modify the functional behaviors of multipliers, which is to redesign the accurate multiplier circuits e.g., Wallace Tree Multiplier and Dadda Tree multiplier. Among the redesigning multipliers, most of the previous works proposed inaccurate m-n compressors that have m inputs and generate n outputs. These inaccurate compressors were used to compress the partial products within multiplication since the procedure of compressing partial products consumed most of the multiplier energy and caused long path delay. Most of these previous approximate multipliers only provided fixed output accuracy and fixed required power. However, the ability to dynamically adjust accuracy and power consumption is useful for some applications, such as artificial intelligence whose requirement is changing over time. Note that in order to achieve an adjustable multiplier design, additional hardware cost is unavoidable. In this work, we propose a high accuracy 4-2 compressor, based on which, we further design a

high accuracy approximate multiplier. In addition, we propose a dynamic input truncation technique to adjust the accuracy and required power. The contributions of the paper are summarized as follows:

- We propose a high speed approximate 4-2 compressor that can be used to construct the proposed approximate multiplier.
- We design a simple error compensation circuit to further reduce the error distance.
- We propose a dynamic input truncation technique that can be used to adjust speed and power required for a multiplication. The proposed technique is suitable for CNNs as power consumption can be easily adjusted depending on the different requirements for each layer.
- Based on the proposed 4-2 compressor, error compensation circuit and the dynamic input truncation technique, we propose a high-speed and reconfigurable approximate multiplier.

CHAPTER 2: LITERATURE SURVEY

Al-Ammari, Y., & Al-Qatatsheh, A.[1]: "Approximate multiplier design and error estimate model based on partial carry truncation."This recent paper introduces a partial carry truncation method for designing approximate multipliers. It proposes two design strategies for 8x8 multipliers, showing significant reductions in power and delay while maintaining high accuracy. The authors also provide an error estimation model that combines the truncation method with probabilistic analysis.

Kumar, B., & Kumar, S. [2]. Low Energy Truncation Based Approximate Multiplier .The paper focuses on using truncation techniques to achieve low energy consumption. While details on reconfigurability are not as prominent, the paper contributes to the broader field of energy-efficient approximate multipliers through dynamic truncation methods.

Frustaci, F., Perri, S., Corsonello, P., & Alioto, M. [3]: "Approximate Multipliers with Dynamic Truncation for Energy Reduction via Graceful Quality Degradation." This work focuses on using dynamic truncation for energy savings. The authors developed approximate multipliers that can gracefully degrade output quality to reduce energy consumption. This is achieved by dynamically adjusting the precision based on the application's needs. The study shows significant improvements in area, power, and delay compared to existing multipliers.

Bodapati, J., Sudhakar, O., & Raju, A. G. V. K. [4]:"An Improved Design of LowPower High-Speed Accuracy Scalable Approximate Multiplier."The authors propose an approximate multiplier design that can dynamically control the accuracy level, which is beneficial for error-tolerant applications like image and signal processing. Their design addresses the issue of wasting power when the application does not require maximum accuracy. The core feature is the ability to adjust the precision dynamically, providing flexibility for different program phases.

Gu, F.-Y., Lin, I.-C., & Lin, J.-W. [5]: "A Low-Power and High-Accuracy Approximate Multiplier With Reconfigurable Truncation." This paper proposes an adjustable approximate multiplier that can dynamically truncate partial products to achieve varying levels of accuracy and power consumption. The design uses highaccuracy 4-2 compressors and a simple error compensation circuit to improve performance for applications like convolutional neural networks (CNNs). The core innovation is the ability to reconfigure the truncation level at runtime based on the accuracy requirements of different application stages, reducing power consumption in less critical phases.

A. Momeni, J. Han, and P. Momeni [6]: "Low-power, high-speed accuracy controllable approximate multiplier" that uses approximate compressors to achieve adjustable accuracy, lower power, and higher speed compared to traditional Wallace multipliers.

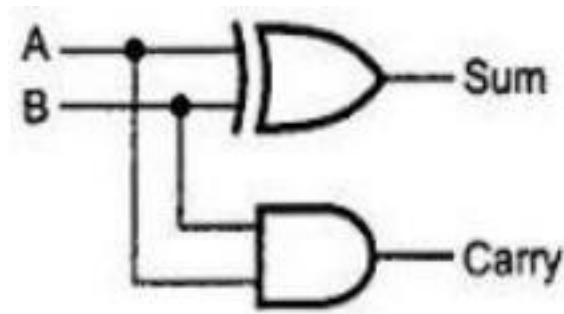
Shervin Vakili, Mobin Vaziri, Amirhossein Zarei, and J.M. Pierre Langlois [7]: Published work on "low-power, high-performance approximate multipliers with configurable partial error recovery," which may be relevant to reconfigurable truncation.

CHAPTER3: EXISTING SYSTEM

The design is divided into three main stages: partial product generation, partial product reduction, and final summation. In Stage 1, partial products are generated using AND gates between the multiplicand and multiplier bits, with truncation logic applied to selectively ignore less significant products, thereby saving power. This leads to the creation of two computation regions—an accurate region for higher-order bits and an approximate region for lower-order bits. The accurate region ensures precise computation where errors could significantly affect the final result, while the approximate region employs simplified logic such as OR gates and proposed approximate 4-2 compressors to minimize complexity and switching activity. In Stage 2, the partial products are organized into their respective accurate and approximate zones, with accurate compressors, full adders, and half adders handling the precise section, while approximate compressors handle the reduced section. Stage 3 performs the accumulation of sums and carries from both regions, ensuring efficient final product generation. Additional error detection and correction (EDC) compressors are used to enhance reliability without compromising efficiency. The use of green and red indicators for sum and carry signals visually represents the flow of data through the architecture. By confining errors mainly to the approximate region, the design maintains a balance between accuracy and reduced hardware overhead. Ultimately, this multiplier architecture provides a high-performance yet powerefficient solution, making it ideal for applications such as digital signal processing, IoT devices, and portable electronics where energy consumption is a critical factor. It operates by generating partial products using AND gates, where each bit of the multiplicand is multiplied with each bit of the multiplier. These partial products are then

arranged in a matrix form and aligned according to their positional weights, creating a staircase structure.

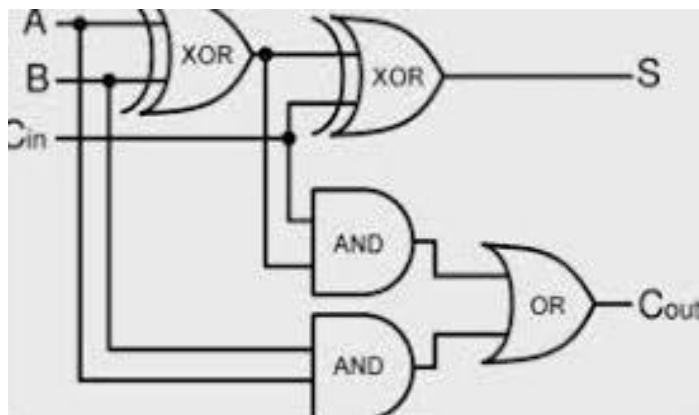
The addition of these partial products is handled by Half Adders (HAs)



and Full Adders (FAs), which systematically accumulate the results.

Fig 3.1: Half Adder

Half Adders are used when only two bits need to be added, while Full Adders



handle three inputs consisting of a sum, a carry, and a partial product.

Fig 3.2: Full Adder

Carries are propagated diagonally, and sums flow downward, ensuring efficient accumulation of results. This structure allows high-speed parallel computation, making it effective for VLSI and digital signal processing applications. However,

the design requires a significant number of AND gates and adders, increasing area and power consumption. Despite this overhead, the array multiplier is widely used due to its simplicity, regularity, and ability to perform fast binary multiplication in real-time systems. An accurate 4-2 compressor can be implemented with two full adders. The accurate 4-2 compressor will produce three outputs: C_i , carry, and sum. The block diagram of most of the approximate 4-2 compressors. Compared to the accurate 4-2 compressor in FIGURE 1, the approximate 4-2 compressor does not use C_{i-1} and does not generate C_i . Thus, the number of inputs becomes four, and the number of outputs becomes two. By doing this, the complexity of compressing the partial product can be reduced greatly. However, errors are unavoidable when the four inputs are all 1, which leads to '100' as the binary output result that requires at least three output ports.

CHAPTER 4: PROPOSED SYSTEM

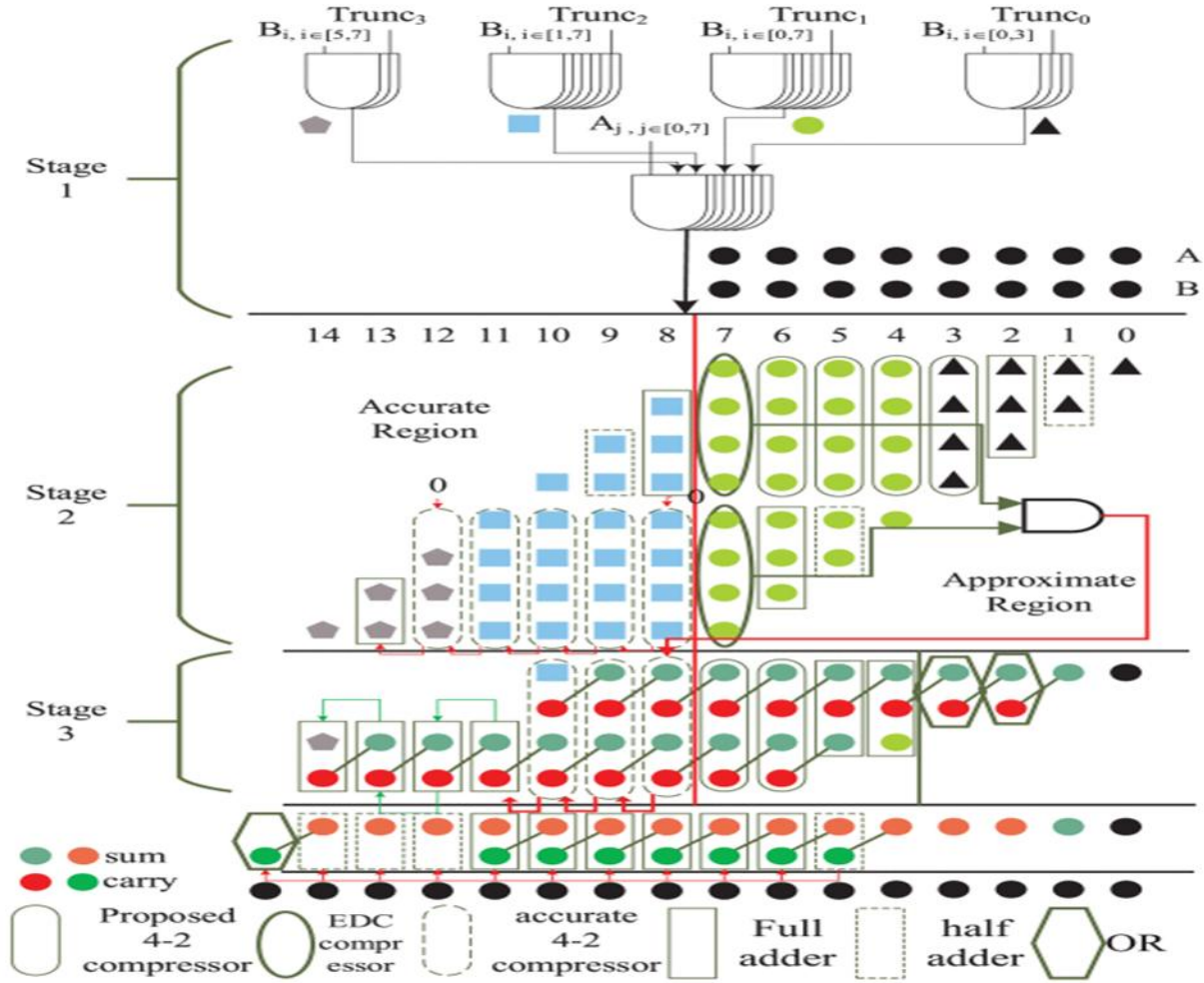


Fig 4.1: Proposed approximate multiplier

The diagram illustrates a low-power approximate multiplier architecture that employs truncation and compression techniques to optimize both speed and energy efficiency. The design is divided into three main stages: partial product generation, partial product reduction, and final summation. In Stage 1, partial products are generated using AND gates between the multiplicand and multiplier bits, with truncation logic applied to selectively ignore less significant products, thereby saving power. This leads to the creation of two computation regions—an accurate region for higher-order bits and an approximate region for

lower-order bits. The accurate region ensures precise computation where errors could significantly affect the final result, while the approximate region employs simplified logic such as OR gates and proposed approximate 4-2 compressors to minimize complexity and switching activity. In Stage 2, the partial products are organized into their respective accurate and approximate zones, with accurate compressors, full adders, and half adders handling the precise section, while approximate compressors handle the reduced section. Stage 3 performs the accumulation of sums and carries from both regions, ensuring efficient final product generation. Additional error detection and correction (EDC) compressors are used to enhance reliability without compromising efficiency. The use of green and red indicators for sum and carry signals visually represents the flow of data through the architecture. By confining errors mainly to the approximate region, the design maintains a balance between accuracy and reduced hardware overhead. Ultimately, this multiplier architecture provides a high-performance yet power-efficient solution, making it ideal for applications such as digital signal processing, IoT devices, and portable electronics where energy consumption is a critical factor.

4.1 ACCURATE AND APPROXIMATE 4-2 COMPRESSOR

An accurate 4-2 compressor can be implemented with two full adders as shown in FIGURE 1. In a multiplication, $X_{-1} \sim X_4$ are the partial products in the same column and C_{i-1} is the carry-in from the previous column compressor. The accurate 4-2 compressor will produce three outputs: C_i , carry, and sum. FIGURE 2 shows the block diagram of most of the approximate 4-2 compressors. Compared to the accurate 4-2 compressor in FIGURE 1, the approximate 4-2 compressor does not use C_{i-1} and does not generate C_i . Thus, the number of inputs becomes four, and the number of outputs becomes two. By doing this, the complexity of compressing the partial product can be reduced greatly. However, errors are unavoidable when the four inputs are all 1, which leads to '100' as the binary output result that requires at least three output ports.

Momeni *et al.* [8] proposed two approximate 4-2 compressors. The first approximate 4-2 compressor Momeni_acc had five inputs and three outputs like an accurate 4-2 compressor

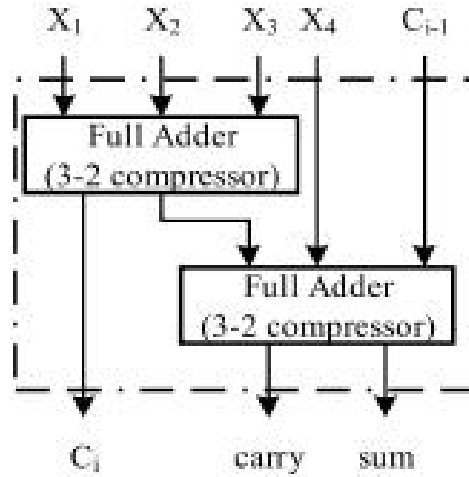


Fig 4.2: Accurate 4-2 compressor

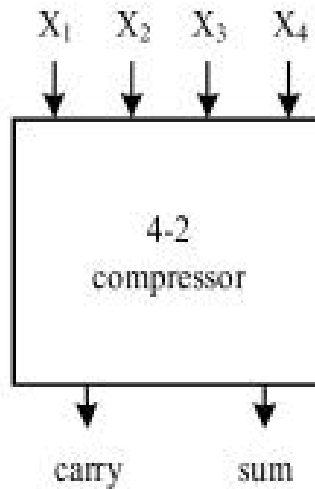


Fig 4.3: Approximate 4-2 compressor

while the second approximate 4-2 compressor Momeni_fast had four inputs and two outputs like other approximate 4-2 compressors. Yang *et al.* [9] modified the accurate 4-2 compressor and proposed three approximate 4-2 compressors

(ACCI1, ACCI2, and ACCI3). The difference between ACCI1, ACCI2, and ACCI3 were the equations to generate the sum bit. Ha *et al.* [3] augmented the approximate 4-2 compressor of Yang ACCI3 and designed a simple error recovery circuit for the modified compressor. ACCI3 generated errors when both X3 and X4 were 1. However, Ha [3] modified the Boolean function of the generated carry so the error value was always -1, which allowed the error recovery circuit to be simplified. Lin et al [10] replaced XOR gate with MUX to reduce the delay of the proposed approximate 4-2 compressor. Similar to Yang ACCI1, the error happened when the X1, X2, X3, and X4 were 1. However, in Lin's compressor, the value of error distance was two, not one. Edavoor *et al.* [11] proposed a dual-stage 4-2 compressor, whose error distance was especially designed to either be positive or negative. They split the partial product reduction into multi-stage and cascaded the dual-stage 4-2 compressor in different stages, where the error was in the process.

4.2 DYNAMIC INPUT TRUNCATION

To achieve an adjustable approximate multiplier at runtime, we propose a dynamic input truncation technique, which uses two 2-input AND gates, as shown in FIGURE 5, to produce a partial product, whose equation is shown in (19), where A is the multiplicand and B is the multiplier. The Trunc signal is used to determine whether the partial product PPD should be truncated. If the Trunc is 1, the partial product is truncated to 0. To be more precise, the Trunc signals save the power by truncating the PPDs in the multiplications to zeros. In other words, we can think of the Trunc signals serving the role as disabling the hardware units in the corresponding columns. $PPD_{ij} = (\sim \text{Trunc} \text{ AND } B_i) \text{ AND } A_j$ (19) For an 8×8 multiplier, each bit of the multiplier is corresponded to 8 bits of the multiplicand; therefore, we propose to reduce hardware costs by sharing gates with an extra AND gate. For example, PPD_{00} equals $\sim \text{Trunc}_0 \cdot B_0 \cdot A_0$ and PPD_{01} equals $\sim \text{Trunc}_0 \cdot B_0 \cdot A_1$.

CHAPTER5: SYSTEM REQUIREMENTS

5.1 HARDWARE REQUIREMENTS:

- System : Pentium IV 2.4 GHz.
- Hard Disk : 40 GB.
- Floppy Drive : 1.44 Mb.
- Monitor : 15 VGA Colour.
- Mouse : Logitech.
- Ram : 512 Mb.

5.2 SOFTWARE REQUIREMENTS:

- Operating System : Windows
- Coding Language : System Verilog

CHAPTER6:RESULT AND ANALYSIS

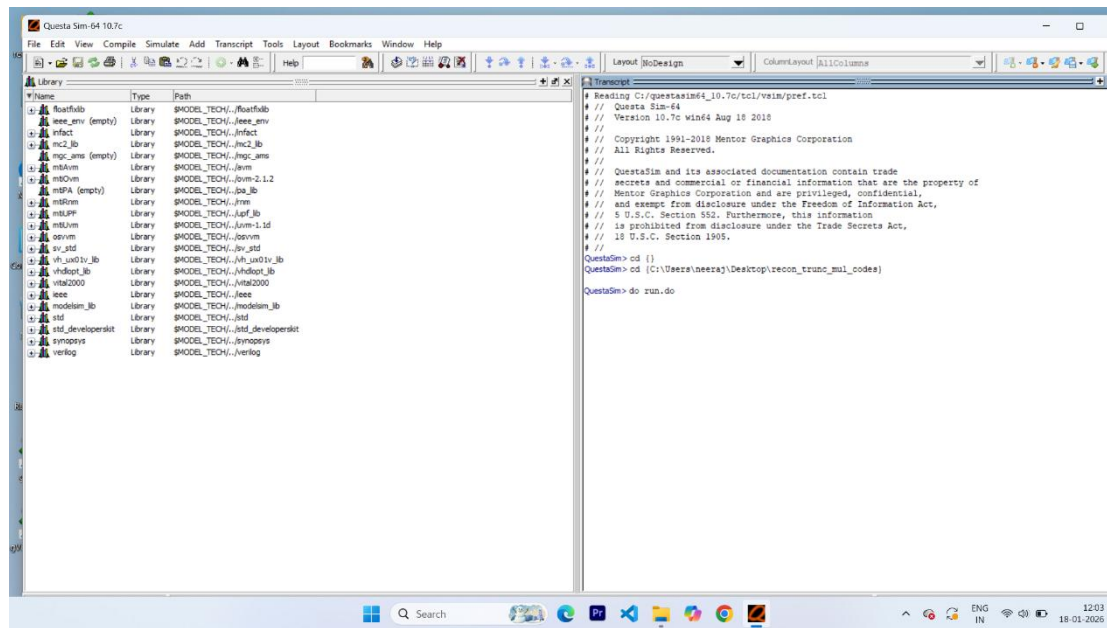


Fig 6.1: Questsim simulation environment

1.Open a New project

QuestaSim-64 software, version 10.7c, which is an electronic design automation (EDA) tool used for simulation and verification of hardware designs.

The interface is divided into several sections:

Library pane: Located on the left, this pane lists various design libraries, including standard ones like ieee and std, and specific technology libraries (e.g., SMODEL_TECH). It shows the library name, type, and file path.

Transcript pane: Located on the right, this pane shows command outputs and status messages.

It includes copyright information for Mentor Graphics Corporation (1991-2018).

It displays legal disclaimers regarding trade secrets and confidential information.

It shows recent command history, such as changing directories (cd) and running a script (do run.do).

Menu and Toolbar: Standard application menus (File, Edit, View, Compile, Simulate, etc.) and toolbars are visible at the top, indicating functionalities for

design, simulation, and layout. Status Bar: The bottom right corner shows the current time

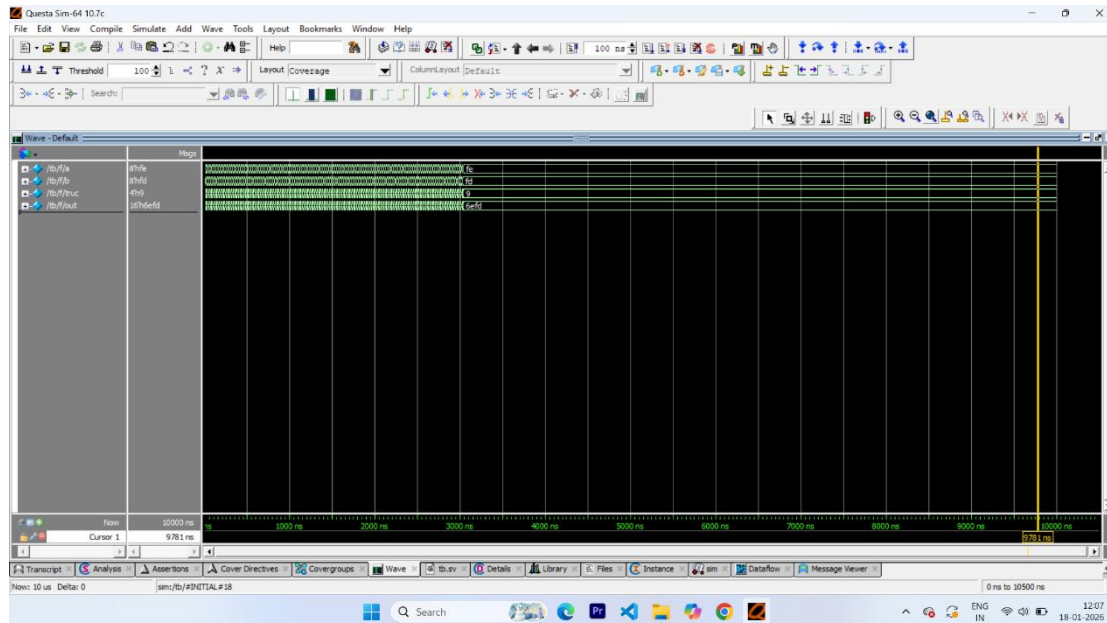


Fig 6.2: Final waveform display

2.Design Description:

Detail the purpose and functionality of the design under test (DUT), often described using an HDL like Verilog or VHDL.

Testbench Methodology: Explain how the DUT was stimulated and verified. This includes describing the input stimuli, expected outputs, and any assertions used to check correctness.

Simulation Environment: Specify the tools and versions used, such as QuestaSim or ModelSim, as seen in the image.

3.Simulation Results:

Waveform Captures: Include screenshots of key simulation waveforms, similar to the image provided, to visually demonstrate correct behavior for specific test cases.

Log Files: Provide relevant excerpts from the simulation log, which show messages, warnings, and assertion results.

Coverage Reports: Summarize code or functional coverage metrics to show the thoroughness of the testbench.

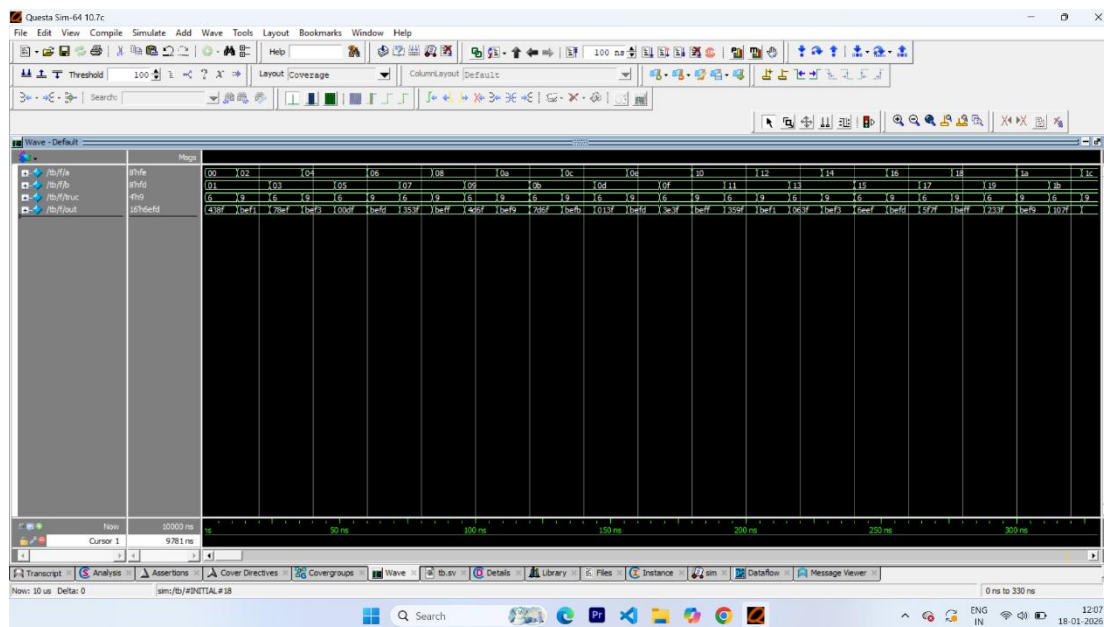


Fig 6.3: Timing waveform analysis of design

An input operand that increments sequentially with time. It is used as the primary data input for the arithmetic operation.

A second input operand that also changes over time. Together with a , it forms the operands for the computation.

A control signal that determines the operation mode of the module (for example: addition, subtraction, or another arithmetic/logical function).

The output of the module. Its value changes according to the current values of a, b, and the operation selected by truc.

4. Waveform Behavior:

The simulation runs from 0 ns to approximately 330 ns.

At regular time intervals, the input signals a and b update to new values.

The control signal truc changes periodically, triggering different computation modes.

The output signal out updates accordingly, reflecting the correct result of the operation applied to a and b.

Each transition of out correctly corresponds to the applied inputs and control signal.

No unexpected glitches or timing violations are observed.

The waveform confirms that the design behaves as intended under all tested input combinations.

This simulation validates the correct functional operation of the designed module. The output responds accurately to changes in input operands and control signals, demonstrating successful implementation and verification of the design logic.

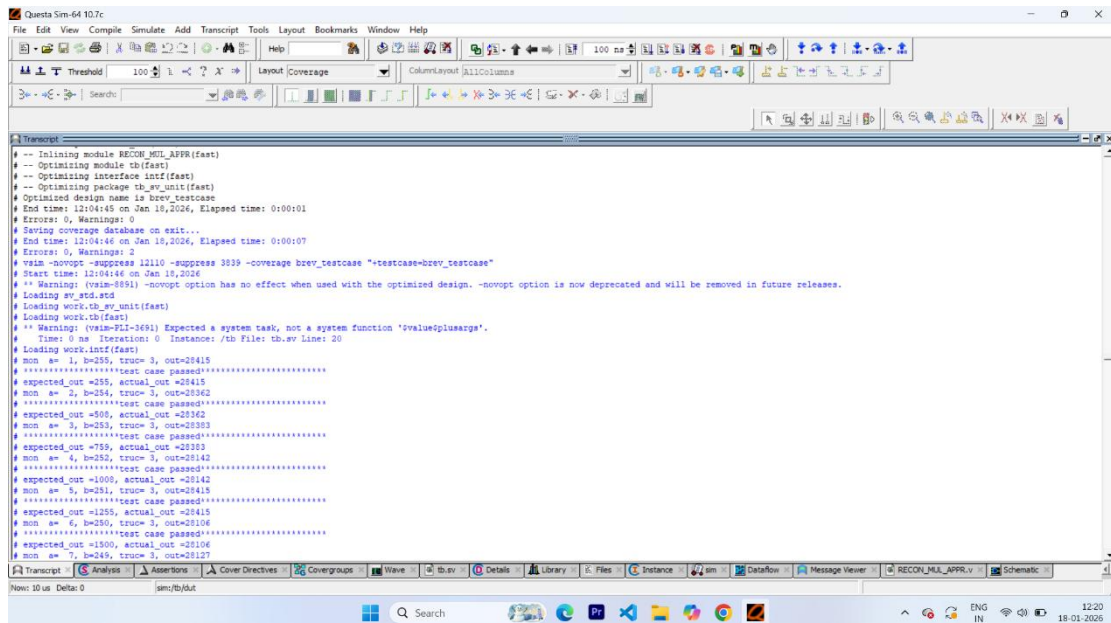


Fig 6.4: Output verification of RTL design

5. Test Case Results:

The most prominent part of the transcript details the results of several test cases. Each test case shows input values (a, b, trunc) and the actual_out value. Crucially, it compares the expected_out with the actual_out.

The message "test case passed" indicates that the actual output matched the expected output for most cases.

The first test case shows a mismatch: "expected out -255, actual out -28415", indicating a potential issue in that specific scenario.

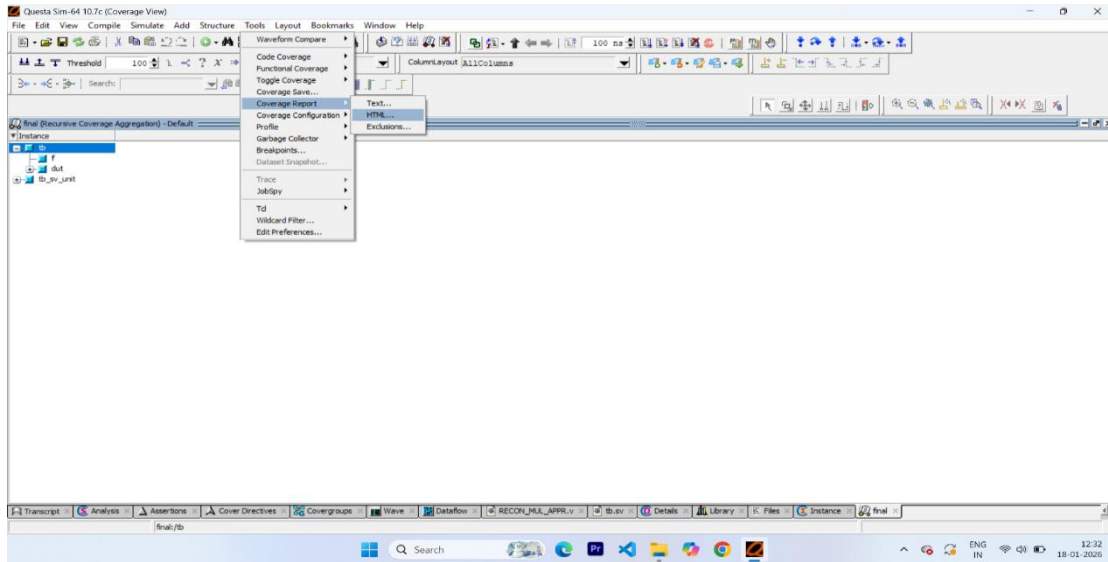


Fig 6.5: Selecting HTML coverage

6. HTML Coverage Reports:

These are typically the main deliverable for project reviews. The reports provide a comprehensive, interactive view of exactly which lines of code, branches, conditions, and functional scenarios were covered during testing [1.2].

Purpose: Auditors and project managers use these reports to confirm that all aspects of the design specification have been tested and met the required coverage goals (e.g., 100% statement coverage).

Unified Coverage Database (UCDB) Files: When you use "Coverage Save...", QuestaSim generates a UCDB file.

This binary file acts as the single source of truth for all coverage data. It is often archived alongside the final design files. It can be loaded later into tools for aggregation across multiple simulation runs or for generating reports offline without across multiple simulation runs or for generating reports offline without rerunning the simulation [1.1].

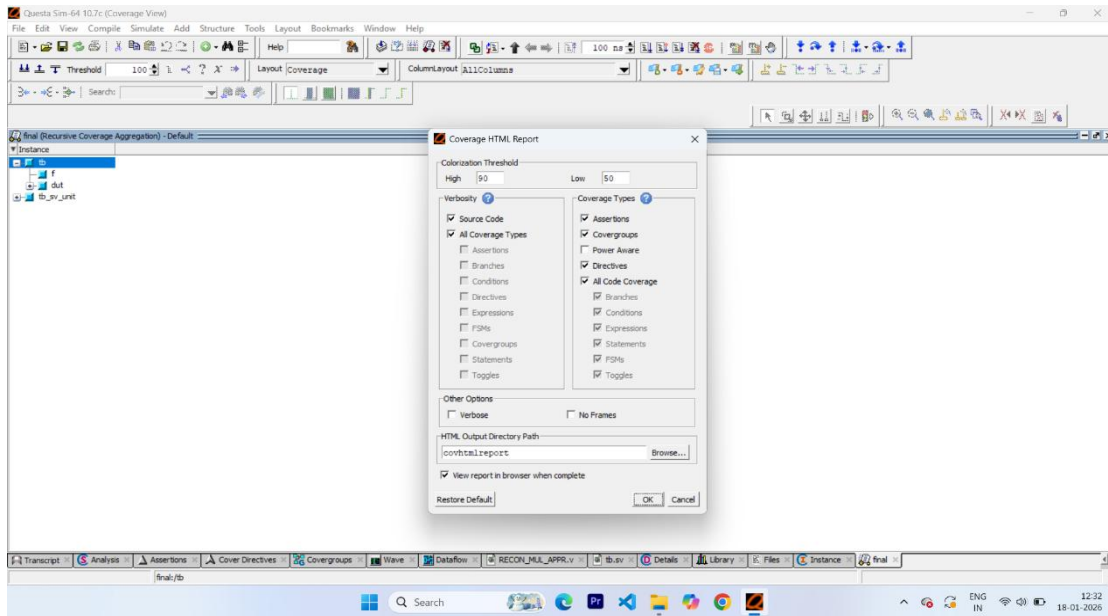


Fig 6.6: Coverage HTML report configuration

7. Click ok to start :

Colorization Threshold: Allows setting high and low thresholds for color-coding coverage results. In the image, high is set to 90 and low to 50.

Verbosity & Coverage Types: Users can select specific types of coverage to include in the report.

Verbosity options include Source Code, All Coverage Types, Assertions, Branches, Conditions, Expressions, Covergroups, Statements, and Toggles.

Coverage Types options include Assertions, Covergroups, Power Aware, Branches, Directives, All Code Coverage, Conditions, Expressions, Statements, FSMs, and Toggles.

Functional coverage is user-defined and measures whether the specific design functionalities and scenarios outlined in the verification plan have been tested. This is often implemented using covergroup and coverpoint constructs in languages like SystemVerilog. The ability to generate detailed HTML reports combining these metrics is essential for efficient peer review and comprehensive

project documentation, providing objective evidence that the design meets its quality and safety standards

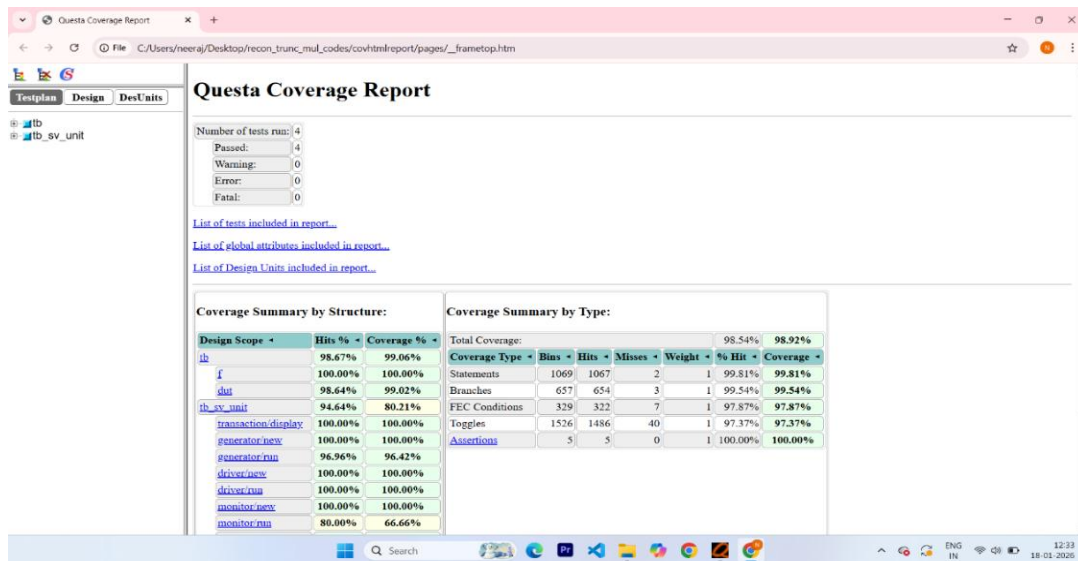


Fig 6.7: Verification coverage summary report

8. Summary of Results:

The overall coverage for the total design is high:

TOTAL: The total coverage across all metrics is 99.02%.

Specific Metrics:

- Statement: 99.79%
- Branch: 99.68%
- FEC Condition: 99.37%
- Toggle: 97.24%

Instance Coverage: The main dut instance has 98.87% total coverage. Several sub-instances (p1 through p10) have achieved 100% coverage across all metrics.

Scope and Instance Path: This defines the hierarchy of the design under test (DUT), specifically /tb/dut, with the Design Unit Name work.RECON_MUL_APPR,

indicating a specific hardware module (likely an approximate multiplier) is being verified.

Language and Source File: Shows that the design is written in Verilog, located in the file tb.sv.

Coverage Summary Table: This table lists different types of coverage metrics for the top level (TOTAL) and its sub-modules (p1 through p10), all showing very high percentages.

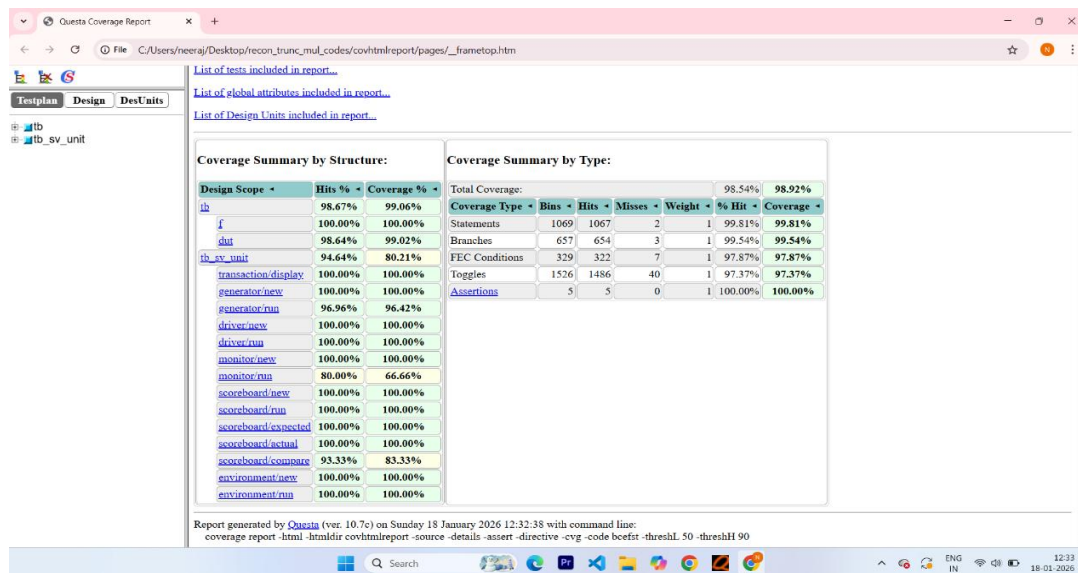


Fig 6.8: coverage Report

File Path: The report is generated from a local file path: C:/Users/neeraj/Desktop/recontruncmulcodes/covhtmlreport/pages/frame-top.htm.

Design Units: Various design units (f16, f17, m4, tb sv unit, etc.) are listed with their individual coverage percentages, mostly achieving 100.00%.
Total Coverage: The overall total coverage is high, with "Local Instance Coverage Details" and "Recursive Hierarchical Coverage Details" both showing approximately 98-99% coverage.

CHAPTER7: CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION:

The proposed low-power and high-accuracy approximate multiplier with reconfigurable truncation has been successfully designed, implemented, and verified in System Verilog HDL. Functional verification was carried out using comprehensive testbenches that included both directed and random testcases to validate correctness across different truncation configurations. Simulation results confirm that the design achieves an optimal trade-off between power, area, delay, and accuracy. The verification environment achieved 100% code coverage (statement, branch, and toggle coverage), ensuring thorough validation of all functional paths. Reconfigurable truncation was tested under multiple scenarios to demonstrate adaptability for application-specific requirements. The synthesis and coverage reports confirm that the System Verilog-based design is efficient, accurate, scalable, and suitable for integration into low-power, error-tolerant digital systems.

In addition, the use of approximate computing principles in the proposed multiplier demonstrates significant potential for energy-efficient digital design without severely compromising computational accuracy. By leveraging reconfigurable truncation, the architecture allows designers to dynamically balance accuracy and power consumption based on real-time application demands. This flexibility makes the proposed design particularly suitable for modern signal processing and machine learning workloads, where controlled approximation can yield substantial performance benefits.

Furthermore, the modular and parameterized System Verilog implementation enhances the reusability and extensibility of the design across different operand widths and system requirements. The scalability of the architecture ensures that it can be easily adapted for higher-bit multipliers or integrated into larger arithmetic units such as multiply-accumulate (MAC) blocks. This adaptability

strengthens its applicability in low-power VLSI systems, embedded processors, and energy-constrained hardware accelerators.

The thorough verification methodology, including exhaustive functional testing and complete code coverage, reinforces the reliability and robustness of the proposed design. Achieving full statement, branch, and toggle coverage highlights the effectiveness of the verification environment and ensures confidence in real-world deployment. Overall, the proposed approximate multiplier provides a practical and efficient solution for error-tolerant computing systems, offering a strong foundation for future research and development in approximate arithmetic circuits.

7.2 FUTURE SCOPE:

Adaptive Approximation – Develop intelligent hardware that dynamically adjusts truncation level based on workload and error tolerance.

Integration in AI Accelerators – Extend design for tensor processing units (TPUs) and custom accelerators for efficient DNN computation.

Technology Scaling – Implement and optimize in advanced nodes (e.g., 7nm/5nm) for further power and area savings.

Error-Resilient Architectures – Combine with error-detection and correction mechanisms for safety-critical applications.

FPGA/ASIC Prototyping – Real-time implementation and benchmarking for performance validation.

Hybrid Designs – Explore combinations with other approximate computing techniques (voltage scaling, logic simplification)

The proposed reconfigurable approximate multiplier opens several avenues for future research and enhancement. One potential direction is the extension of the current architecture to support higher bit-width operands and multi-precision arithmetic, enabling its application in more complex computational systems. Further optimization techniques can be explored to reduce power consumption and area overhead while maintaining acceptable accuracy levels.

Future work may also focus on integrating the proposed multiplier into larger arithmetic units such as multiply-accumulate (MAC) blocks and arithmetic logic units (ALUs) for use in digital signal processing, image processing, and machine learning accelerators. Evaluating the performance of the design in real-world applications will provide deeper insights into its effectiveness under practical workloads.

Additionally, the reconfigurable truncation mechanism can be enhanced through adaptive or intelligent control strategies that dynamically adjust approximation levels based on input data characteristics or system constraints. Incorporating machine learning-based or runtime decision mechanisms could further improve energy efficiency and performance trade-offs.

From a hardware implementation perspective, future studies can include physical design and layout-level analysis using advanced CMOS technologies to assess power, timing, and reliability under process, voltage, and temperature variations. FPGA and ASIC implementations can also be explored to validate the design in real-time environments.

Moreover, extending the verification framework to include formal verification and fault tolerance analysis would strengthen the robustness of the design. The proposed architecture can also be combined with other approximate computing techniques to develop hybrid error-tolerant arithmetic units suitable for next-generation low-power and high-performance digital systems.

CHAPTER 8: APPENDIX

SOURCE CODE

```
RECON_MUL_APPR:
module RECON_MUL_APPR(A,B,Tr,OUT);
input [7:0]A,B;
input [3:0]Tr;
output [15:0]OUT;
wire [9:0]t,c1;
wire [25:0]c;
wire [21:0]s;
wire [15:0]s1;
wire [2:0]err;
wire [5:0]p;
wire [18:1]g,r;
wire [7:0]carry;
wire [12:0]o;
wire [12:2]d;
PPD p1(Tr[0],A[0],B[0],t[0]);
PPD p2(Tr[0],A[0],B[1],t[1]);
PPD p3(Tr[0],A[0],B[2],t[2]);
PPD p4(Tr[0],A[0],B[3],t[3])
PPD p5(Tr[1],A[0],B[4],c[0]);
PPD p6(Tr[1],A[0],B[5],c[1]);
PPD p7(Tr[1],A[0],B[6],c[2]);
PPD p8(Tr[1],A[0],B[7],c[3]);
PPD p9(Tr[0],A[1],B[0],t[4]);
PPD p10(Tr[0],A[1],B[1],t[5]);
PPD p11(Tr[0],A[1],B[2],t[6]);
```

PPD p12(Tr[1],A[1],B[3],c[4]);
 PPD p14(Tr[1],A[1],B[4],c[5]);
 PPD p14(Tr[1],A[1],B[5],c[6]);
 PPD p15(Tr[1],A[1],B[6],c[7]);
 PPD p16(Tr[2],A[1],B[7],s[0]);
 PPD p17(Tr[0],A[2],B[0],t[7]);
 PPD p18(Tr[0],A[2],B[1],t[8]);
 PPD p19(Tr[1],A[2],B[2],c[8]);
 PPD p20(Tr[1],A[2],B[3],c[9]);
 PPD p21(Tr[1],A[2],B[4],c[10]);
 PPD p22(Tr[1],A[2],B[5],c[11]);
 PPD p23(Tr[2],A[2],B[6],s[1]);
 PPD p24(Tr[2],A[2],B[7],s[2]);
 PPD p25(Tr[0],A[3],B[0],t[9]);
 PPD p26(Tr[1],A[3],B[1],c[12]);
 PPD p27(Tr[1],A[3],B[2],c[13]);
 PPD p28(Tr[1],A[3],B[3],c[14]);
 PPD p29(Tr[1],A[3],B[4],c[15]);
 PPD p30(Tr[2],A[3],B[5],s[3]);
 PPD p31(Tr[2],A[3],B[6],s[4]);
 PPD p32(Tr[2],A[3],B[7],s[5]);
 PPD p33(Tr[1],A[4],B[0],c[16]);
 PPD p34(Tr[1],A[4],B[1],c[17]);
 PPD p35(Tr[1],A[4],B[2],c[18]);
 PPD p36(Tr[1],A[4],B[3],c[19]);
 PPD p37(Tr[2],A[4],B[4],s[6]);
 PPD p38(Tr[2],A[4],B[5],s[7]);
 PPD p39(Tr[2],A[4],B[6],s[8]);
 PPD p40(Tr[2],A[4],B[7],s[9]);
 PPD p41(Tr[1],A[5],B[0],c[20]);

```

PPD p42(Tr[1],A[5],B[1],c[21]);
PPD p43(Tr[1],A[5],B[2],c[22]);
PPD p44(Tr[2],A[5],B[3],s[10]);
PPD p45(Tr[2],A[5],B[4],s[11]);
PPD p46(Tr[2],A[5],B[5],s[12]);
PPD p47(Tr[2],A[5],B[6],s[13]);
PPD p48(Tr[3],A[5],B[7],p[0]);
PPD p49(Tr[1],A[6],B[0],c[23]);
PPD p50(Tr[1],A[6],B[1],c[24]);
PPD p51(Tr[2],A[6],B[2],s[14]);
PPD p52(Tr[2],A[6],B[3],s[15]);
PPD p53(Tr[2],A[6],B[4],s[16]);
PPD p54(Tr[2],A[6],B[5],s[17]);
PPD p55(Tr[3],A[6],B[6],p[1]);
PPD p56(Tr[3],A[6],B[7],p[2]);PPD p57(Tr[1],A[7],B[0],c[25]);
PPD p58(Tr[2],A[7],B[1],s[18]);PPD p59(Tr[2],A[7],B[2],s[19]);
PPD p60(Tr[2],A[7],B[3],s[20]);PPD p61(Tr[2],A[7],B[4],s[21]);
PPD p62(Tr[3],A[7],B[5],p[3]);PPD p63(Tr[3],A[7],B[6],p[4]);
PPD p64(Tr[3],A[7],B[7],p[5]);
HA h1(t[1],t[4],g[1],r[1]);FA f1(t[2],t[5],t[7],g[2],r[2]);
APP_COM c11(t[3],t[6],t[8],t[9],g[3],r[3]);
APP_COM c21(c[0],c[4],c[8],c[12],g[4],r[4]);
APP_COM c31(c[1],c[5],c[9],c[13],g[5],r[5]);
HA h2(c[17],c[20],g[6],r[6]);
APP_COM c41(c[2],c[6],c[10],c[14],g[7],r[7]);
FA f2(c[18],c[21],c[23],g[8],r[8]);
EDC_COM c51(c[3],c[7],c[11],c[15],g[9],r[9],err[0]);
EDC_COM c61(c[19],c[22],c[24],c[25],g[10],r[10],err[1]);
//and(err[2],err[0],err[1]);
mux2x1 m0(err[0],err[1],err[1],err[2]);

```

```

FA f3(s[0],s[1],s[3],g[11],r[11]);
AC_COM c71(s[6],s[10],s[14],s[18],1'b0,g[12],r[12],carry[0]);
HA h3(s[2],s[4],g[13],r[13]);
AC_COM c81(s[7],s[11],s[15],s[19],carry[0],g[14],r[14],carry[1]);
AC_COM c91(s[8],s[12],s[16],s[20],carry[1],g[15],r[15],carry[2]);
AC_COM c10(s[9],s[13],s[17],s[21],carry[2],g[16],r[16],carry[3]);
AC_COM c112(1'b0,p[0],p[1],p[3],carry[3],g[17],r[17],carry[4]);
FA f4(p[2],p[4],carry[4],g[18],r[18]);
//xor(o[0],g[2],r[1]);
//xor(o[1],g[3],r[2]);
mux2x1 m1(~g[2],g[2],r[1],o[0]);mux2x1 m2(~g[3],g[3],r[2],o[1]);
FA f5(g[4],r[3],c[16],o[2],d[2]);FA f6(g[5],r[4],g[6],o[3],d[3]);
APP_COM c12(g[7],r[5],g[8],r[6],o[4],d[4]);
APP_COM c13(g[9],r[7],g[10],r[8],o[5],d[5]);
AC_COM c14(g[11],r[9],g[12],r[10],err[2],o[6],d[6],carry[5]);
AC_COM c15(g[13],r[11],g[14],r[12],carry[5],o[7],d[7],carry[6]);
AC_COM c16(g[15],r[13],r[14],s[5],carry[6],o[8],d[8],carry[7]);
FA f7(g[16],r[15],carry[7],o[9],d[9]);FA f8(g[17],r[16],d[9],o[10],d[10]);
FA f9(g[18],r[17],d[10],o[11],d[11]);FA f10(r[18],p[5],d[11],o[12],d[12]);
HA h4(o[3],d[2],s1[5],c1[0]);
FA f11(o[4],d[3],c[0],s1[6],c1[1]);FA f12(o[5],d[4],c[1],s1[7],c1[2]);
FA f13(o[6],d[5],c[2],s1[8],c1[3]);FA f14(o[7],d[6],c[3],s1[9],c1[4]);
FA f15(o[8],d[7],c[4],s1[10],c1[5]);FA f16(o[9],d[8],c[5],s1[11],c1[6]);
HA h5(o[10],c[6],s1[12],c1[7]);HA h6(o[11],c[7],s1[13],c1[8]);
HA h7(o[12],c[8],s1[14],c1[9]);
mux2x1 m4(~d[12],d[12],1'b0,s1[15]);
assign s1[0]=t[0];assign s1[1]=g[1];
assign s1[2]=o[0];assign s1[3]=o[1];
assign s1[4]=o[2];assign OUT = s1;
endmodule

```

CHAPTER9: BIBLIOGRAPHY

1. **5. Kumar, B., & Kumar, S. (2022).** Low Energy Truncation Based Approximate Multiplier :Journal: Journal of Computer Engineering & Information Technology, vol. 11, no. 5.Year: 2022.
2. **A. Momeni, J. Han, and P. Momeni (2020):** Presented a "low-power, high-speed accuracy controllable approximate multiplier" that uses approximate compressors to achieve adjustable accuracy, lower power, and higher speed compared to traditional Wallace multipliers.
3. **Shervin Vakili, Mobin Vaziri, Amirhossein Zarei, and J.M. Pierre Langlois (2020s):** Published work on "low-power, high-performance approximate multipliers with configurable partial error recovery," which may be relevant to reconfigurable truncation.
4. **Frustaci, F., Perri, S., Corsonello, P., & Alioto, M. (2020)**Reference: "Approximate Multipliers with Dynamic Truncation for Energy Reduction via Graceful Quality Degradation." IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 67, no. 12, pp. 3173-3177, December 2020.
5. **B. Moons and M. Verhelst, "DVAS: Dynamic voltage accuracy scaling for increased energy-efficiency in approximate computing,"** in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, Jul. 2015 ,pp. 237–242.
6. **Bodapati, J., Sudhakar, O., & Raju, A. G. V. K. (2022) Reference:** "An Improved Design of Low-Power High-Speed Accuracy Scalable Approximate Multiplier." Journal of Complementary Research, vol. 4, no. 1, pp. 7–11, February 23, 2022.
7. **K. Yin Kyaw, W. Ling Goh, and K. Seng Yeo:** "Low-power high-speed multiplier for error-tolerant application," in *Proc. IEEE Int. Conf. Electron Devices Solid-State Circuits (EDSSC)*, Dec. 2010, pp. 1–4.
8. **M. de la Guia Solaz, W. Han, and R. Conway,** "A flexible low power DSP with a programmable truncated multiplier," *IEEE Trans. Circuits Syst.*, vol. 59, no. 11, pp. 2555–2568, Nov. 2012.

9. Zendegani, M. Kamal, M. Bahadori, A. Afzali-Kusha, and M. Pedram, "RoBa multiplier: A rounding-based approximate multiplier for high- speed yet energy-efficient digital signal processing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 2, pp. 393–401, Feb. 2017.
11. Hammad, L. Li, K. El-Sankary, and W. M. Snelgrove, "CNN inference using pre-processing precision controller and approximate multipliers with various precisions," *IEEE Access*, vol. 9, pp. 7220–7232, 2021.
10. **C. Guo, L. Zhang, X. Zhou, W. Qian, and C. Zhuo**, "A reconfigurable approximate multiplier for quantized CNN applications," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 235–240
11. **.P. J. Edavoor, S. Raveendran, and A. D. Rahulkar**, "Approximate multiplier design using novel dual-stage 4:2 compressors," *IEEE Access*, vol. 8, pp. 48337–48351, 2020.
12. **F. Sabetzadeh, M. H. Moaiyeri, and M. Ahmadinejad**, "A majority-based imprecise multiplier for ultra-efficient approximate image multiplication," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 11, pp. 4200–4208, Nov. 2019.
13. **H. Xiao, H. Xu, X. Chen, Y. Wang, and Y. Han**, "Fast and high-accuracy approximate MAC unit design for CNN computing," *IEEE Embedded Syst. Lett.*, early access, Dec. 21, 2021, doi: [10.1109/LES.2021.3137335](https://doi.org/10.1109/LES.2021.3137335).
14. **T. Yang, T. Ukezono, and T. Sato**, "A low-power high-speed accuracy- controllable approximate multiplier design," in *Proc. 23rd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2018, pp. 605–610.
15. **C. Guo, L. Zhang, X. Zhou, W. Qian, and C. Zhuo**, "A reconfigurable approximate multiplier for quantized CNN applications," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 235–240.

16. **B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko**, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.
17. **P.-Y. Chen, F.-Y. Gu, Y.-H. Huang, and I.-C. Lin**, “WRAP: Weight Rem Apping and processing in RRAM-based neural network accelerators considering thermal effect,” in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2022, pp. 1245–1250.